

Aula 5

- *Timers*
 - Aplicações
 - Princípio de funcionamento
 - Divisão de frequência e controlo do *duty-cycle*
- *Watchdog timer*
- Estudo de caso: *Timers* do PIC32
 - Estrutura e funcionamento
 - Geração de sinais PWM (*output compare module*)

José Luís Azevedo, Bernardo Cunha, Tomás O. Silva, P. Bartolomeu

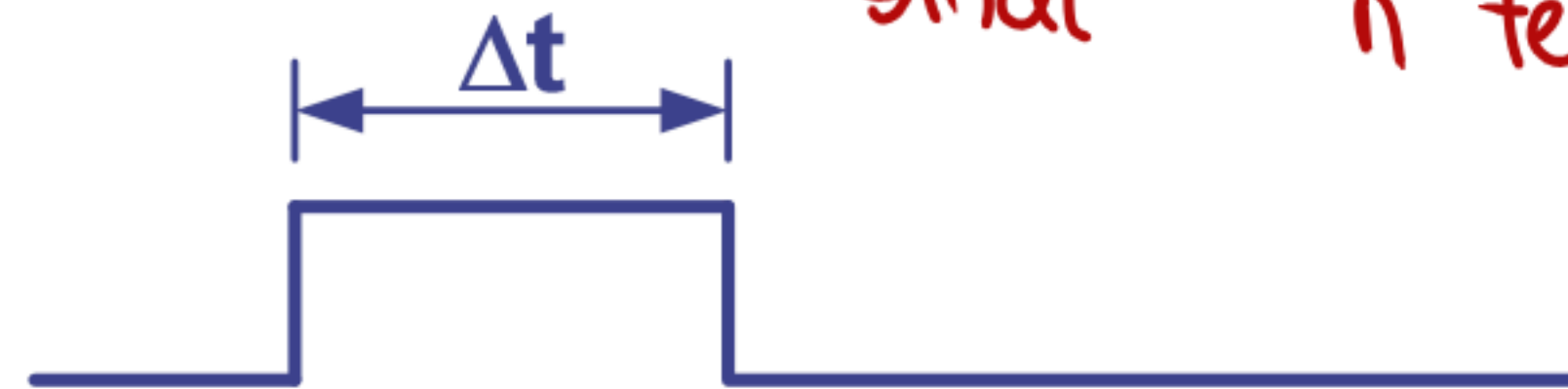
Introdução

hardware

- Um *timer* é um dispositivo periférico de suporte que permite, no essencial, a medição de tempo, partindo de uma referência temporal conhecida
- Alguns exemplos de aplicações típicas de *timers*:

- geração de um evento com uma duração controlada;

exemplo: geração de um impulso com uma duração definida, Δt



sinal n tem de ser periodico

- geração de um evento periódico com período controlado;

exemplo: geração de um impulso com um período definido, T

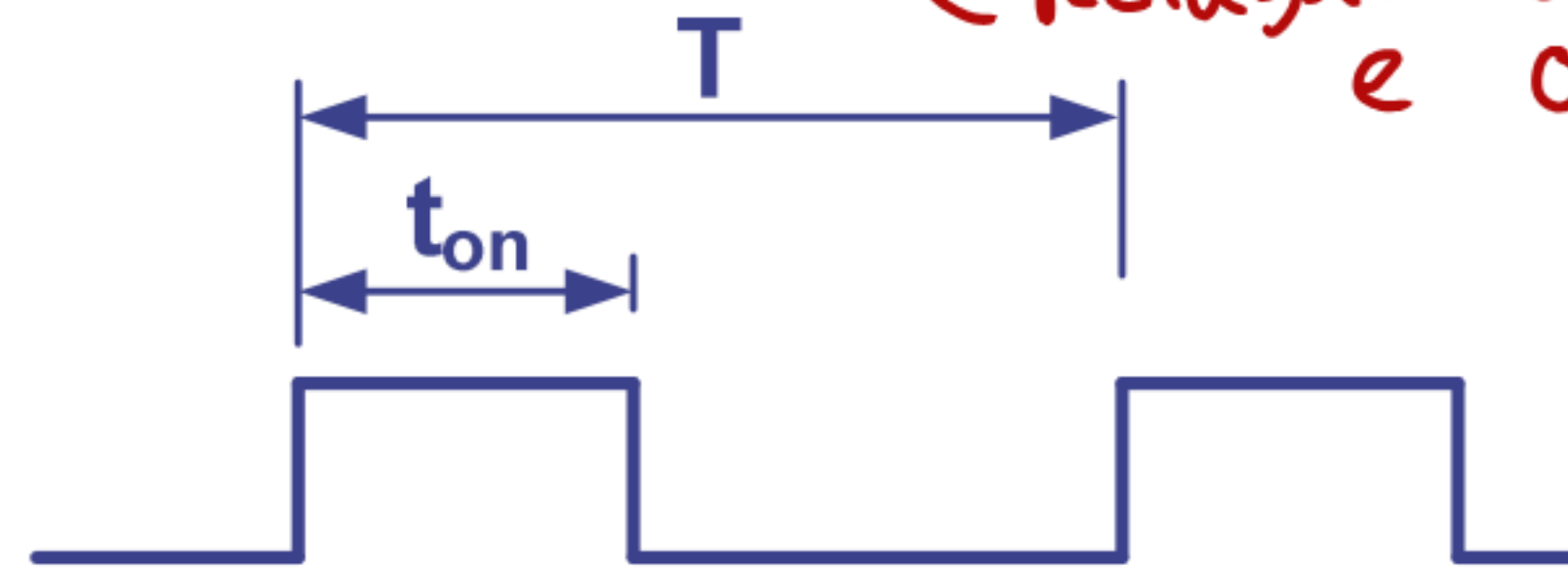


sinal de n em n tempo

• n dá por escolher quanto tempo fica a 1

Introdução

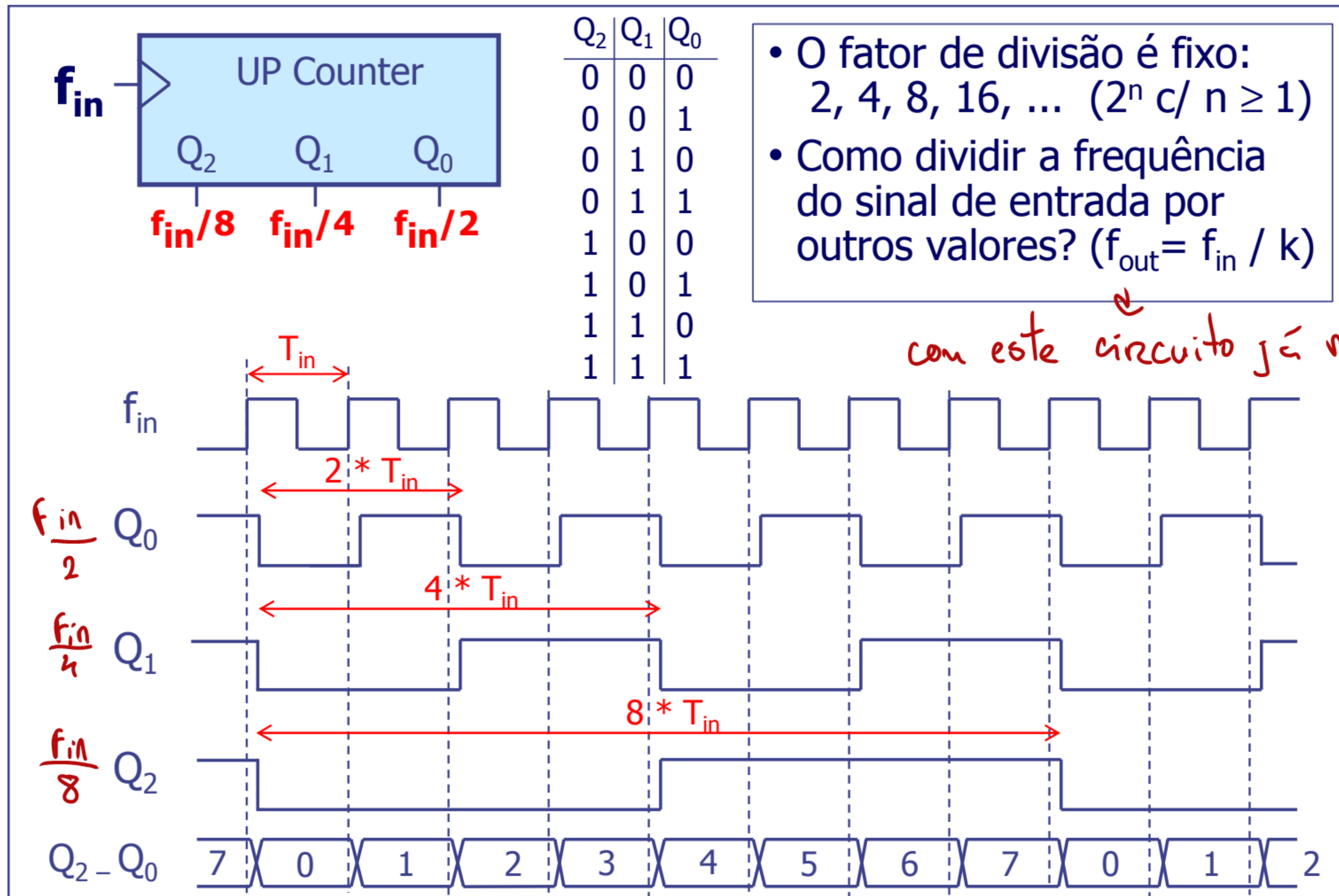
- Tem de se ter sempre uma referência temporal - sinal de relógio com freq. conhecida
- Geração de um evento periódico com período e duração controlados. Exemplo: geração de um sinal periódico com um período de 10 ms e um "duty-cycle" de 40%:
relação entre o tempo a 1 e o período



$$\text{Duty-cycle} = (t_{\text{on}} / T) * 100 \text{ [\%]}$$

- "ton" é o tempo durante o qual o sinal está no nível lógico 1, num período
- a possibilidade de alterar o valor de "ton" sem alterar o valor de T é útil em muitas situações e designa-se por **PWM** (**Pulse Width Modulation** – modulação por largura de pulso)
- O funcionamento dos *timers* baseia-se sempre na contagem de ciclos de um sinal de relógio com frequência conhecida

Divisão de frequência



- O fator de divisão é fixo:
2, 4, 8, 16, ... (2^n c/ $n \geq 1$)
- Como dividir a frequência do sinal de entrada por outros valores? ($f_{out} = f_{in} / k$)

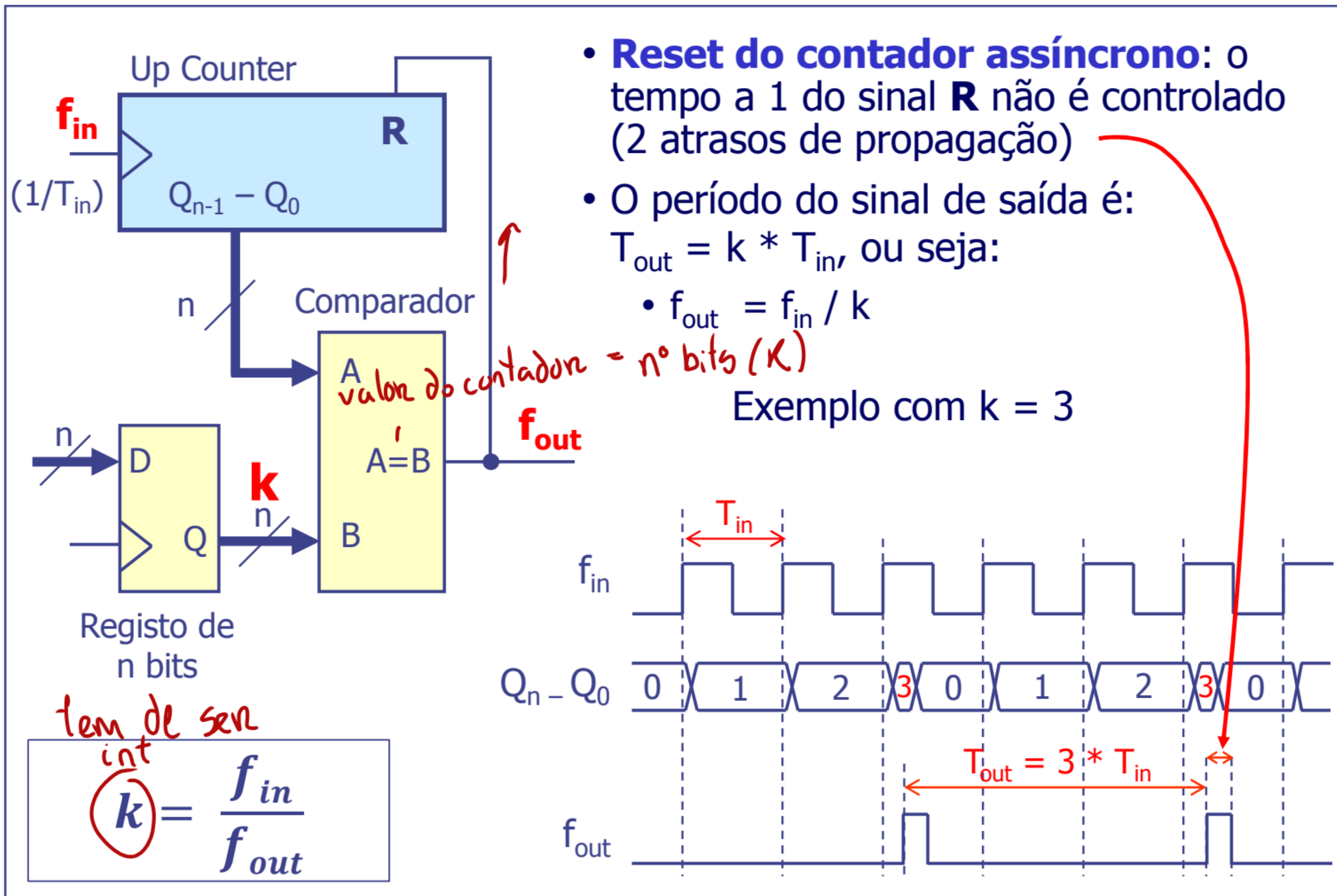
Solução:

Divisão de frequência (versão 1)

- **Reset do contador assíncrono:** o tempo a 1 do sinal **R** não é controlado (2 atrasos de propagação)
- O período do sinal de saída é:
 $T_{out} = k * T_{in}$, ou seja:
 - $f_{out} = f_{in} / k$

valor do contador = nº bits (K)

Exemplo com $k = 3$



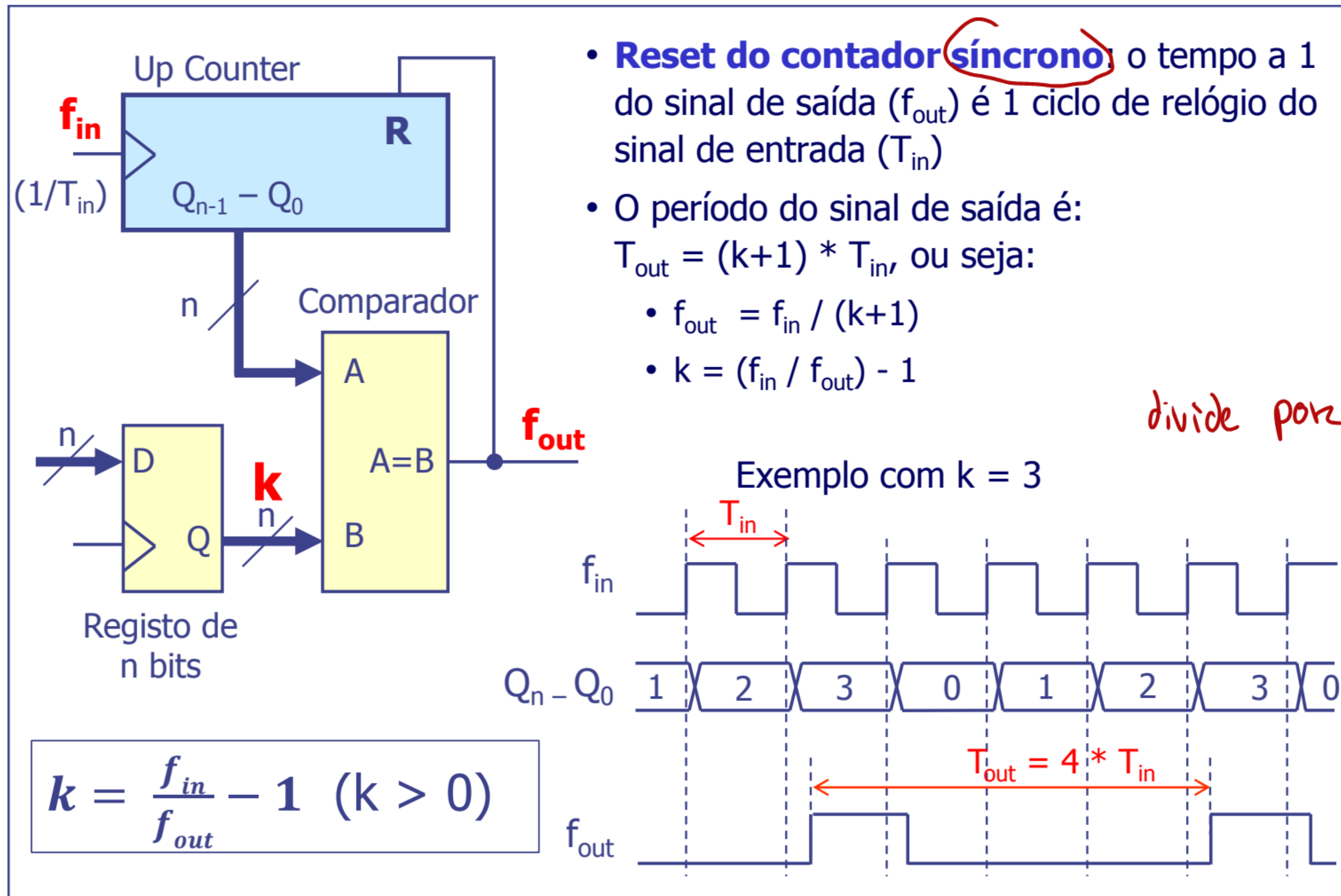
tem de ser

$$k = \frac{f_{in}}{f_{out}}$$

$$f_{out} = \frac{f_{in}}{k}$$

Divisão de frequência (versão 2)

única diferença

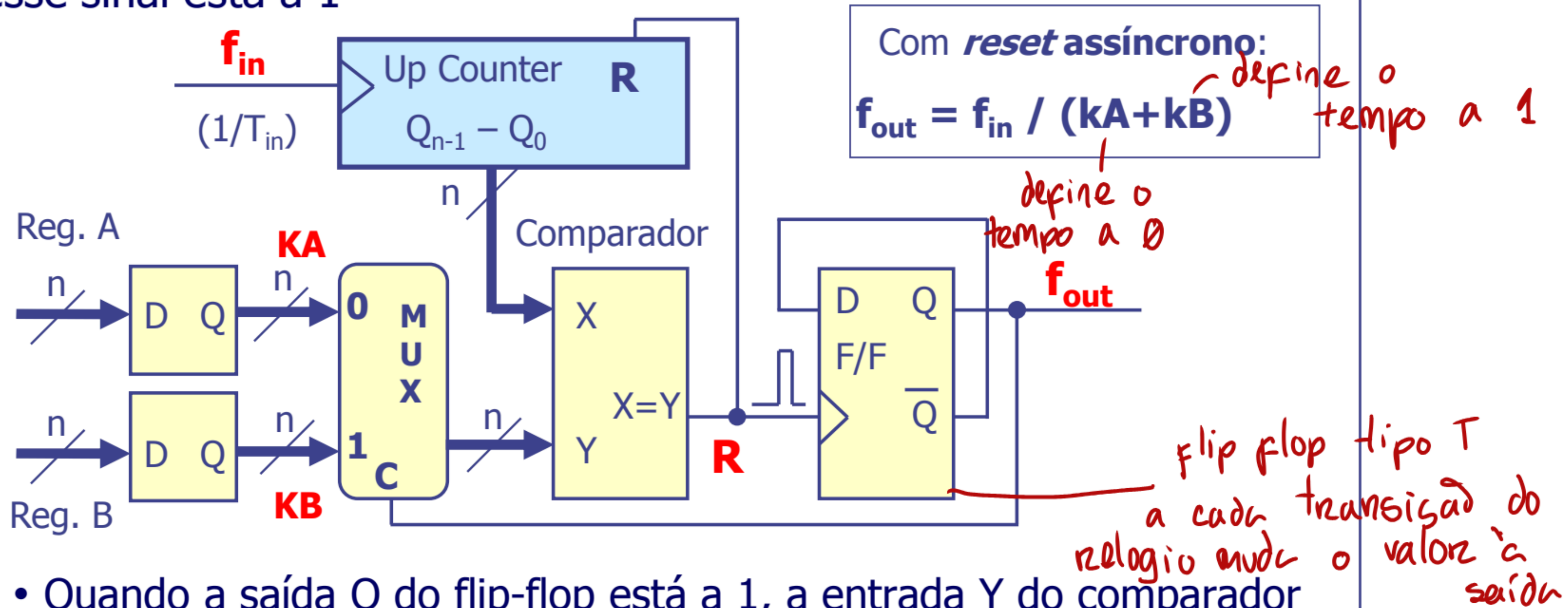


- **Reset do contador síncrono:** o tempo a 1 do sinal de saída (f_{out}) é 1 ciclo de relógio do sinal de entrada (T_{in})
- O período do sinal de saída é:
 $T_{out} = (k+1) * T_{in}$, ou seja:
 - $f_{out} = f_{in} / (k+1)$
 - $k = (f_{in} / f_{out}) - 1$

divide por $k+1$

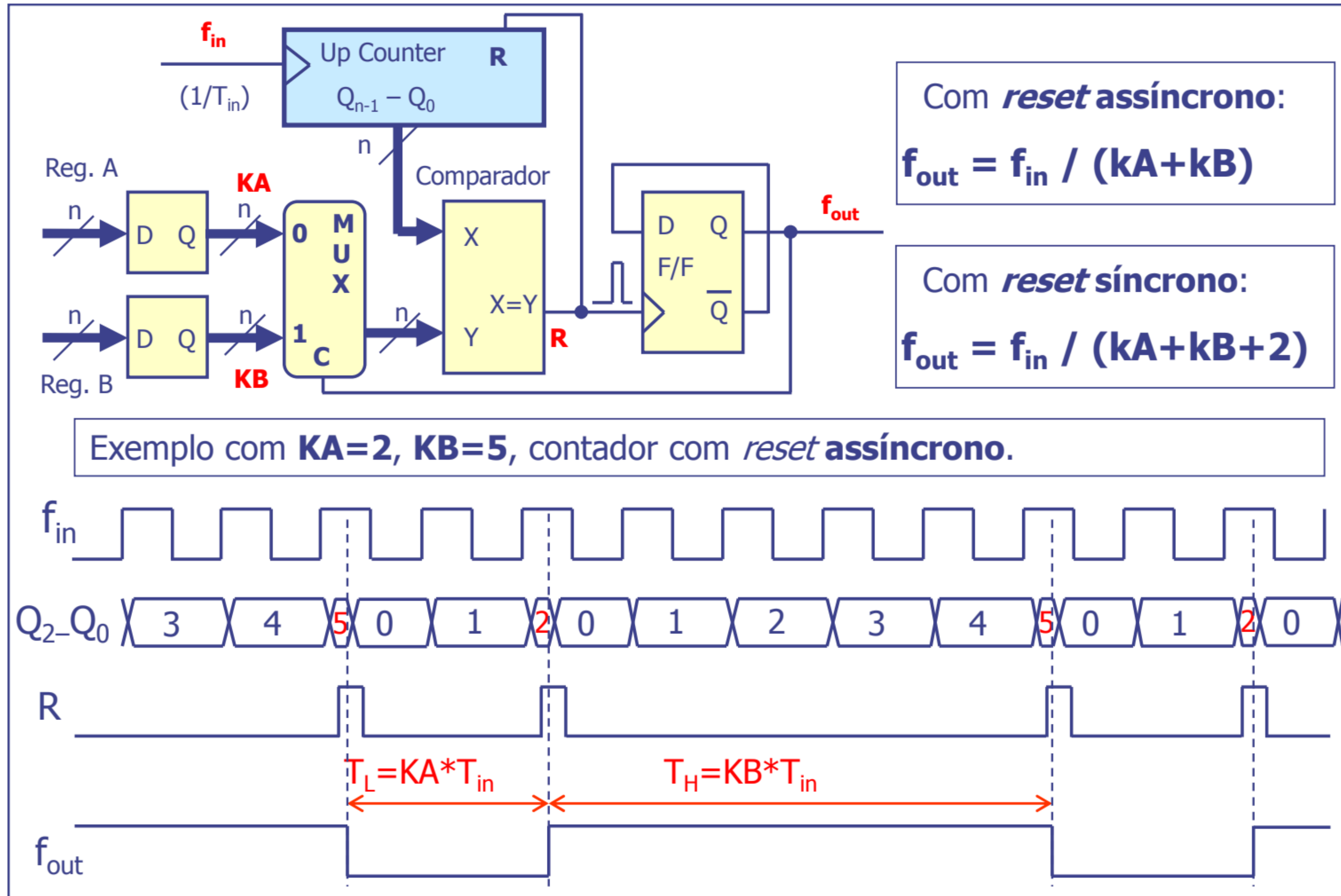
Controlo de frequência e *duty-cycle* - controlo do tempo a 1

- Para este tipo de aplicação, o timer tem que permitir o controlo simultâneo do período do sinal de saída e do tempo durante o qual esse sinal está a 1



- Quando a saída Q do flip-flop está a 1, a entrada Y do comparador toma o valor de KB ; caso contrário, toma o valor de KA . Logo, o tempo durante o qual o sinal de saída está a 1 depende de KB , e durante o qual está a 0 depende de KA

Controlo de frequência e *duty-cycle*

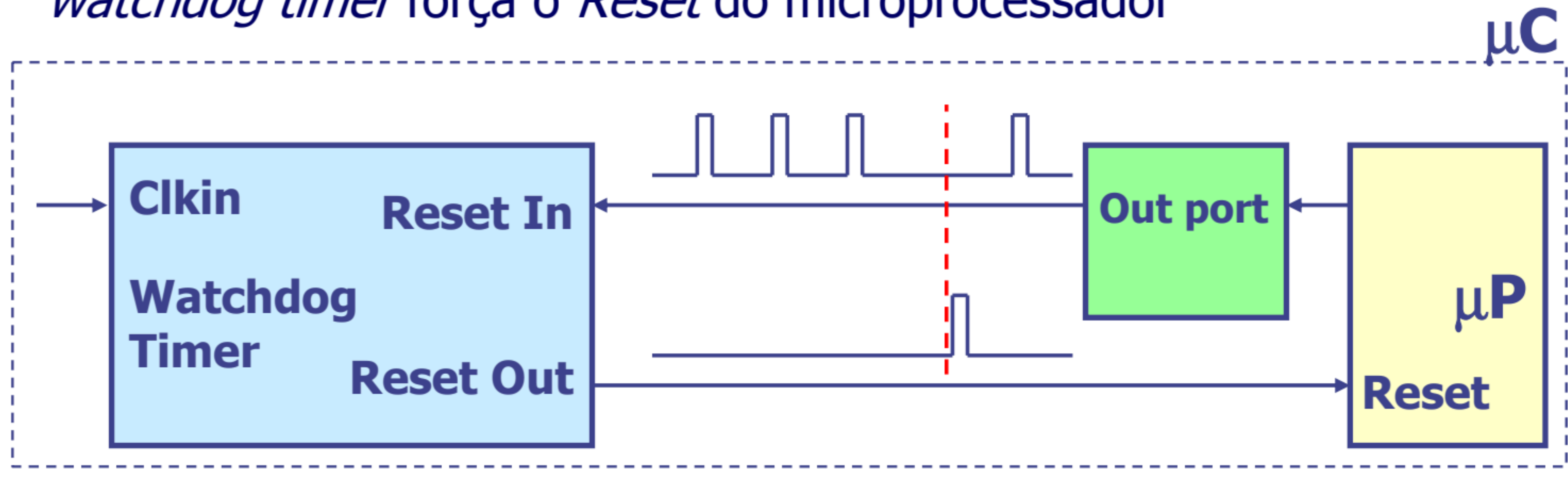


modar duty cycle sem mudar periodo $\rightarrow$ aumentar KA e diminuir KB (vice-versa)

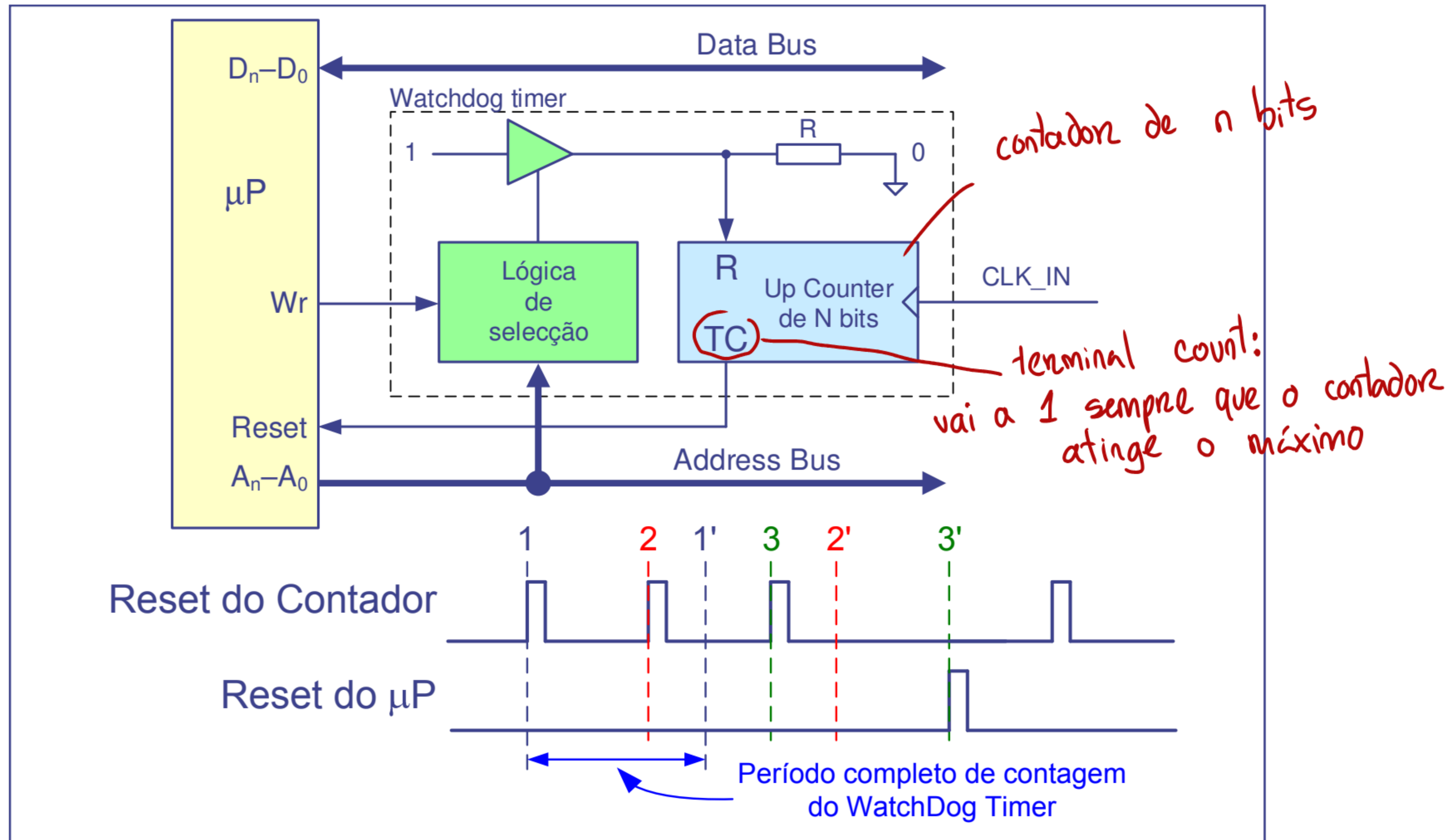
Watchdog Timer (temporizador "cão de guarda")

tipo dead man switch

- Sistemas baseados em microcontrolador podem assegurar funções de controlo críticas que não podem falhar
- Como garantir que um crash do microprocessador não compromete o funcionamento global do sistema?
- Um *watchdog timer* tem como função monitorizar a operação do microprocessador e, em caso de falha, forçar o seu reinício
- Situação mais comum: se o microprocessador não atuou a entrada de *Reset* do *watchdog timer* ao fim de um tempo pré-determinado o *watchdog timer* força o *Reset* do microprocessador



Watchdog Timer – exemplo de implementação



Watchdog Timer – exemplo de utilização

↪ *estão ativados por default*

- A aplicação no microcontrolador executa em ciclo infinito
- O *watchdog timer* é ativado quando o programa inicia. O *reset* da sua contagem é feito regularmente no corpo do ciclo (no exemplo, `clearWatchdogTimer()`)

```
void main(void)
{
    enableWatchdogTimer();
    (...)
    while(1)
    {
        (...)
        clearWatchdogTimer();
    }
}
```

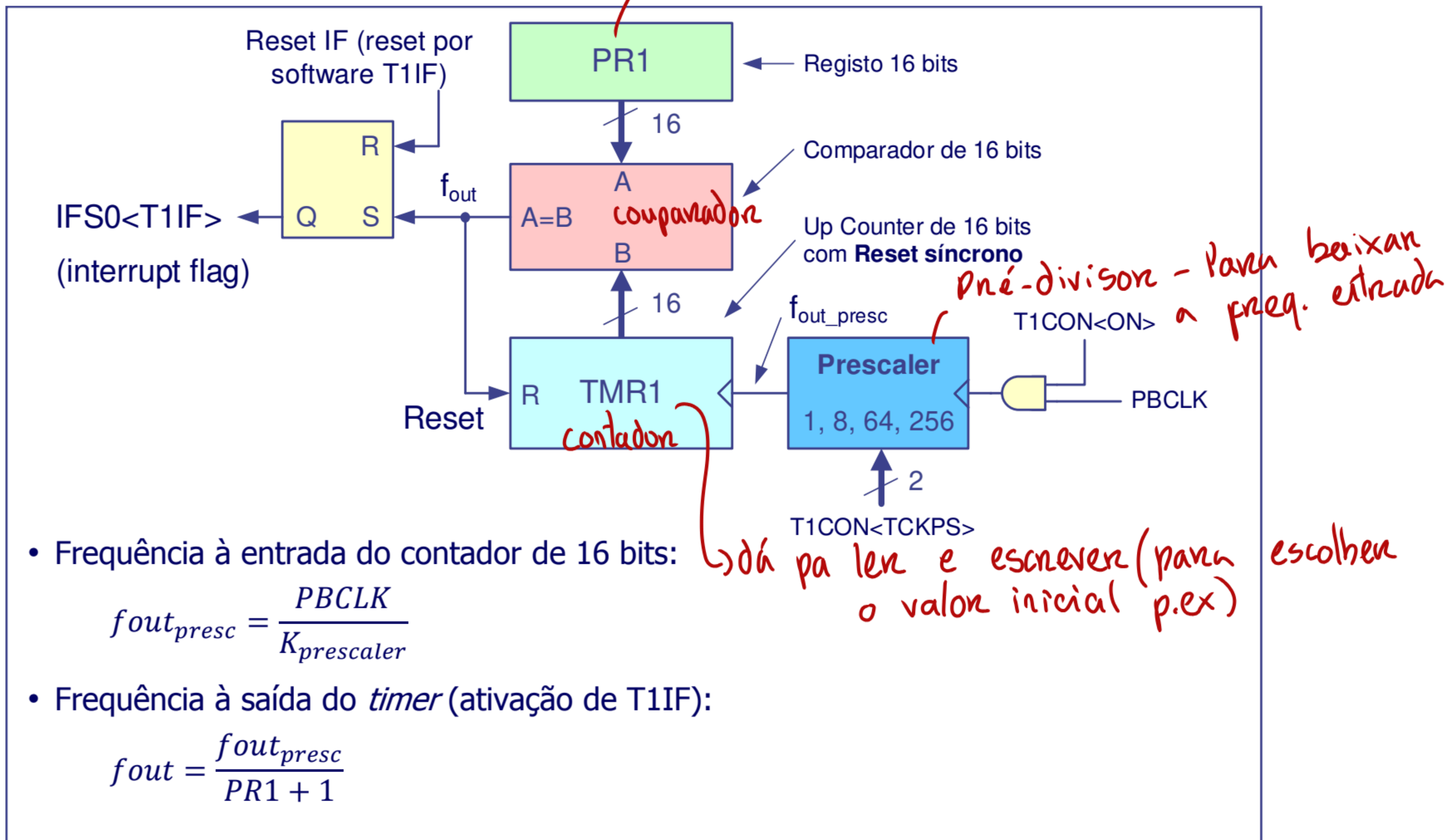
- Caso haja uma falha no microprocessador que implique a quebra de execução do ciclo, a função "clearWatchdogTimer()" deixa de ser chamada e o *watchdog timer* deixa de ser reiniciado:
 - Quando o contador do *watchdog timer* atinge o valor máximo da contagem força um *reset* ao microprocessador

- Estudo de caso: *Timers* do PIC32
 - Estrutura e funcionamento
 - Geração de sinais PWM (*output compare module*)

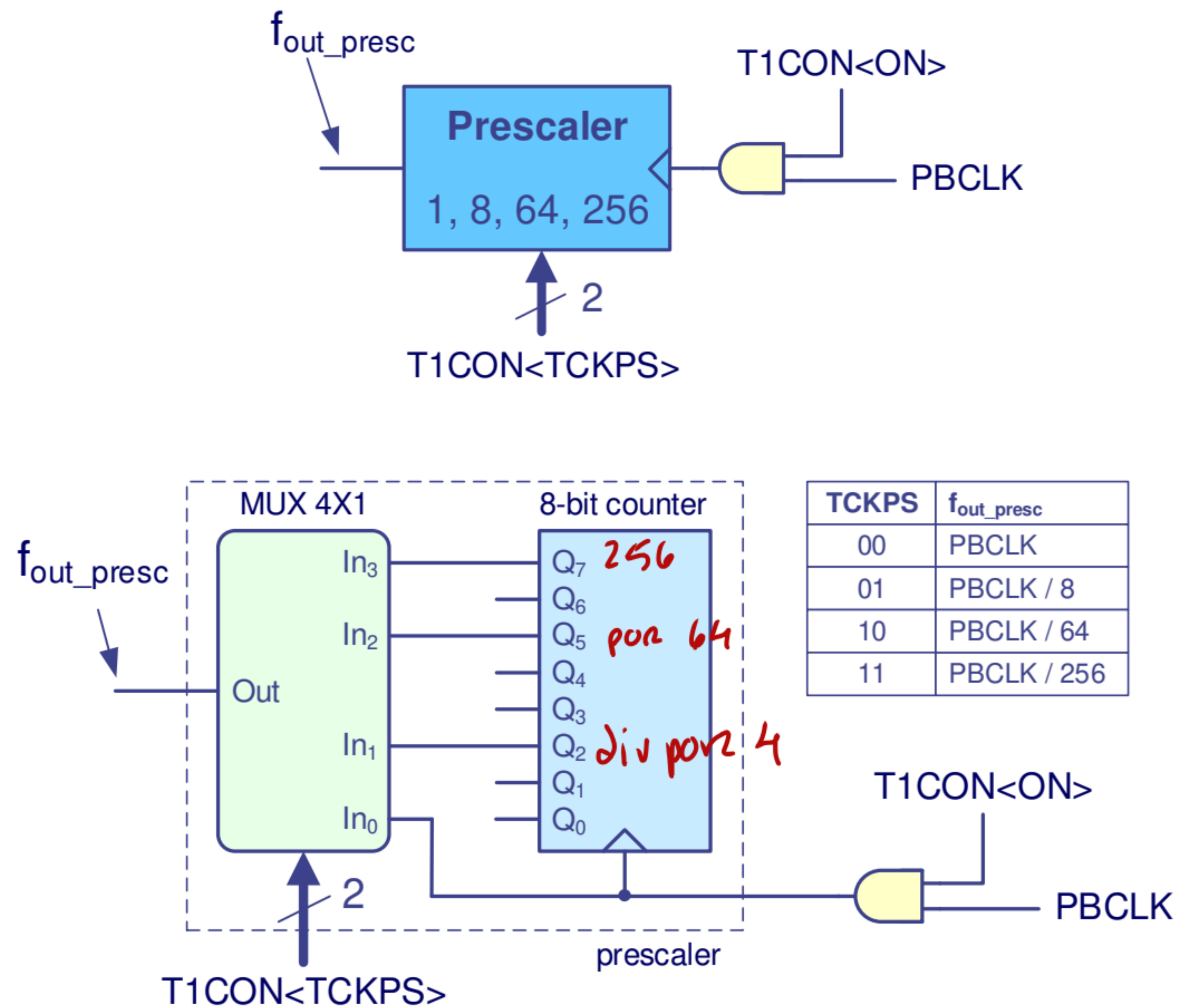
Estudo de caso: *Timers* do PIC32

- A série PIC32MX7xx disponibiliza **5 *timers* de 16 bits**, designados por **T1, T2, T3, T4** e **T5**
contadores e comparadores também
- T2, T3, T4 e T5 têm a mesma estrutura e apresentam o mesmo modelo de programação. São designados pelo fabricante como ***timers* tipo B**
- T2 a T5 podem ser agrupados 2 a 2 para formar 2 *timers* de 32 bits (T2 e T3 e/ou T4 e T5)
- O T1 é designado como ***timer* tipo A**; tem uma estrutura semelhante aos restantes e pequenas diferenças no modelo de programação
- A frequência-base de entrada para os *timers* é dada pelo *Peripheral Bus Clock* (**PBCLK**). Na placa DETPIC32 a frequência de PBCLK é metade da frequência de CPU, i.e. **PBCLK = 20 MHz**
- Os *timers* do PIC32 não têm saída acessível no exterior. Podem ser usados para gerar interrupções (todos) ou como base de tempo para a geração de sinais com "duty-cycle" configurável (T2 e T3)

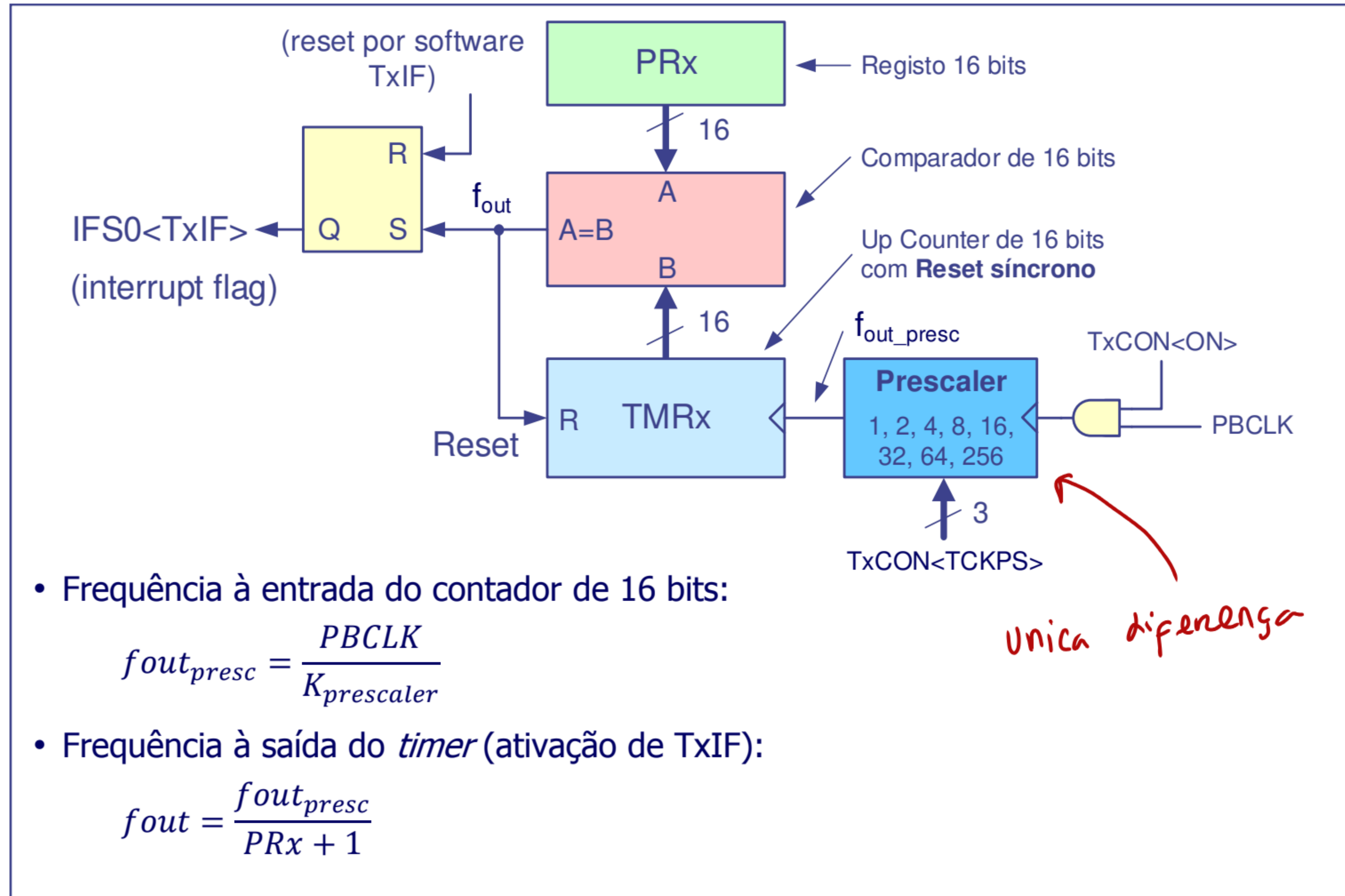
PIC32 – *timer* tipo A (T1)



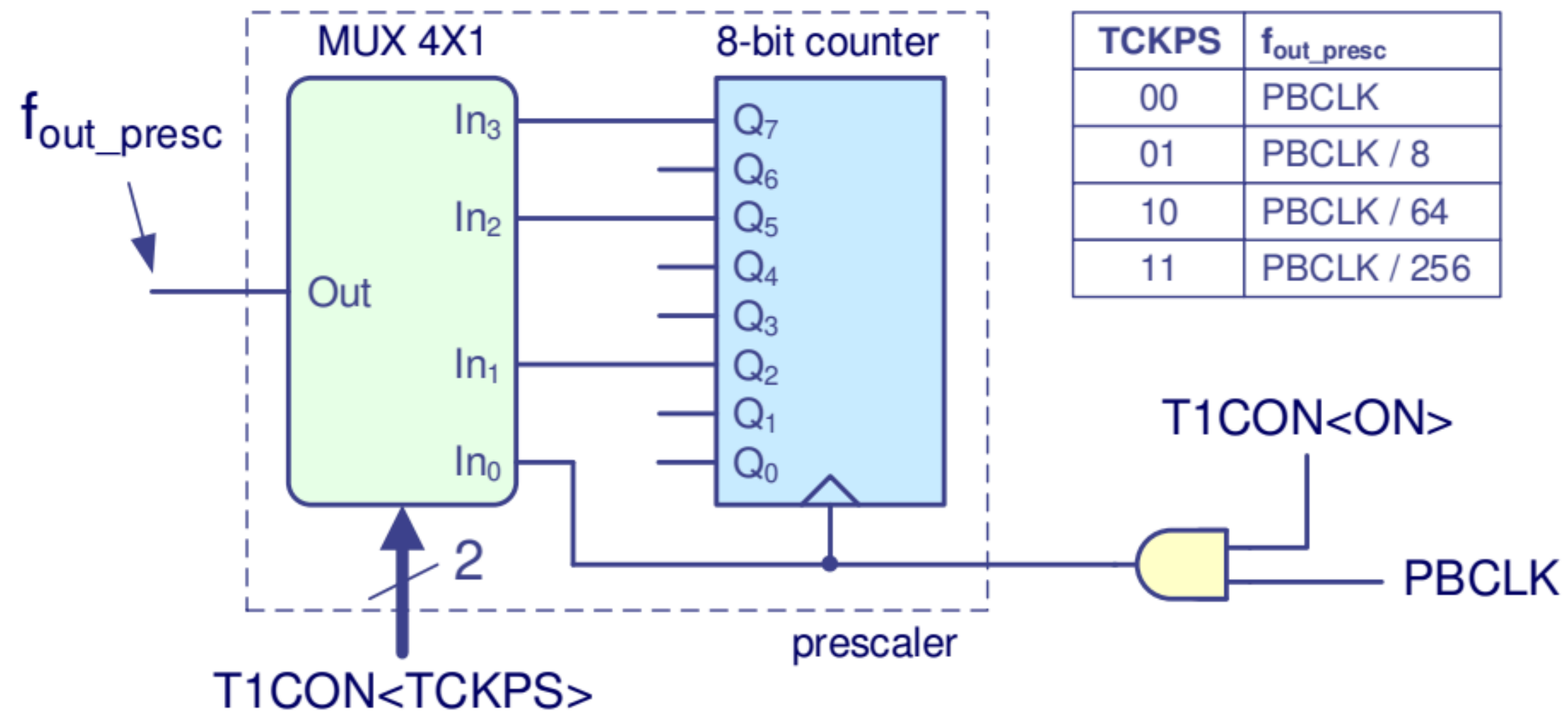
PIC32 – *timer tipo A (prescaler)*



PIC32 – *timers* tipo B (T2 a T5)

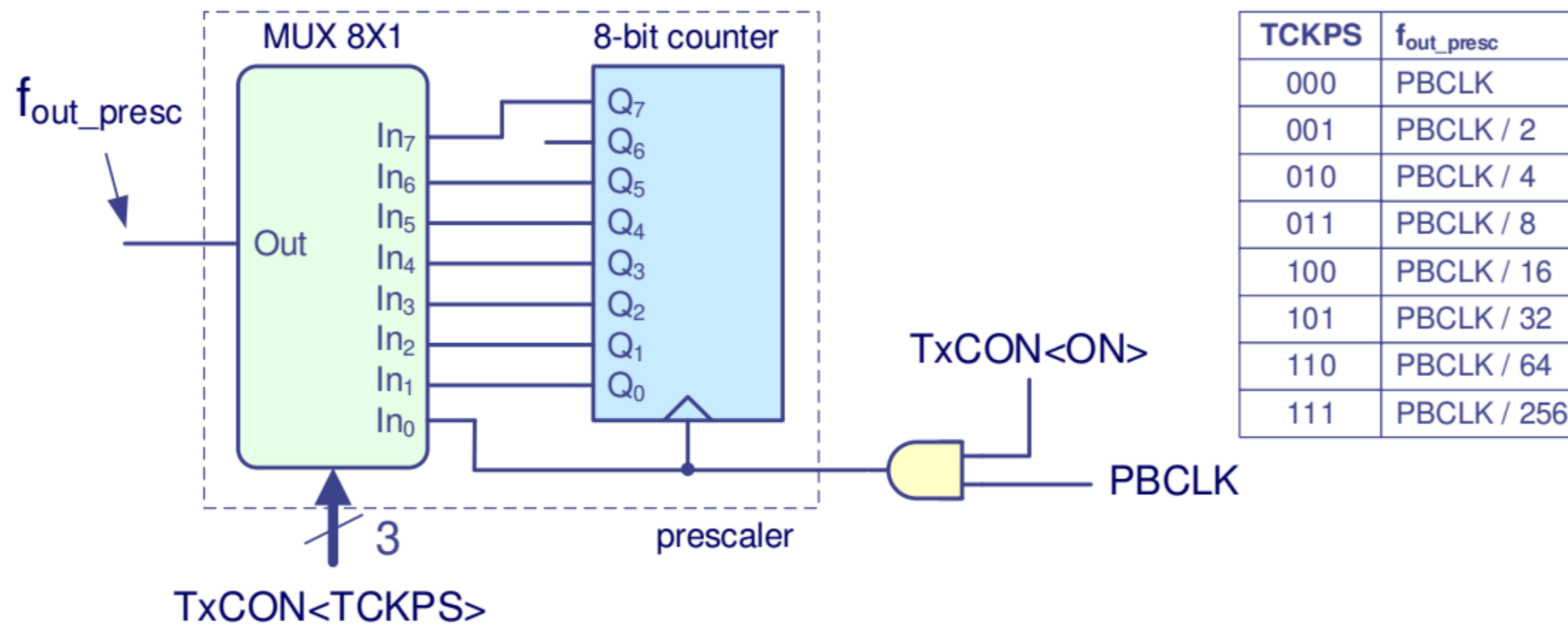


PIC32 –prescalers

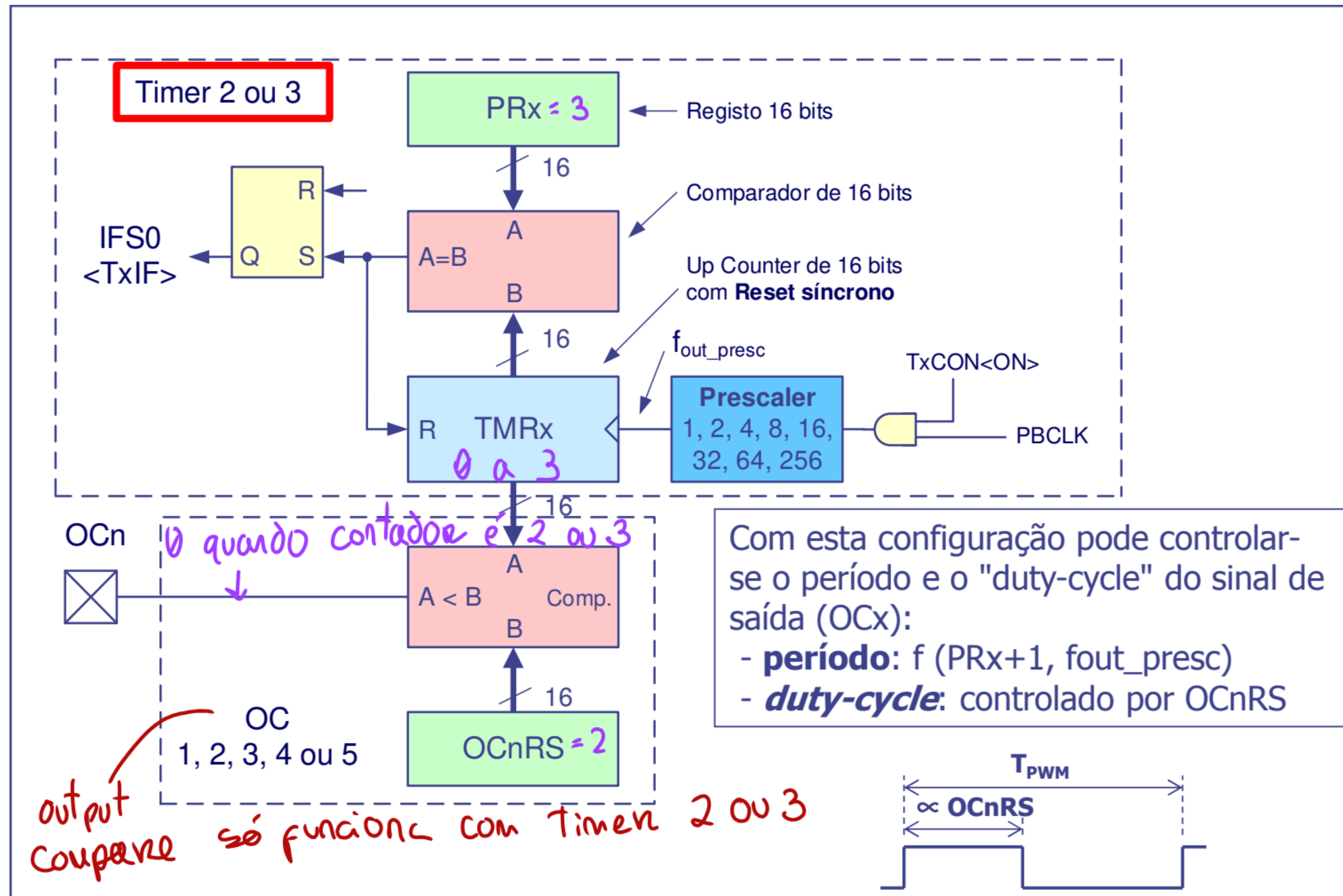


prescaler tipo A

prescaler tipo B



PIC32 – controlo de período e "duty-cycle" *ex*



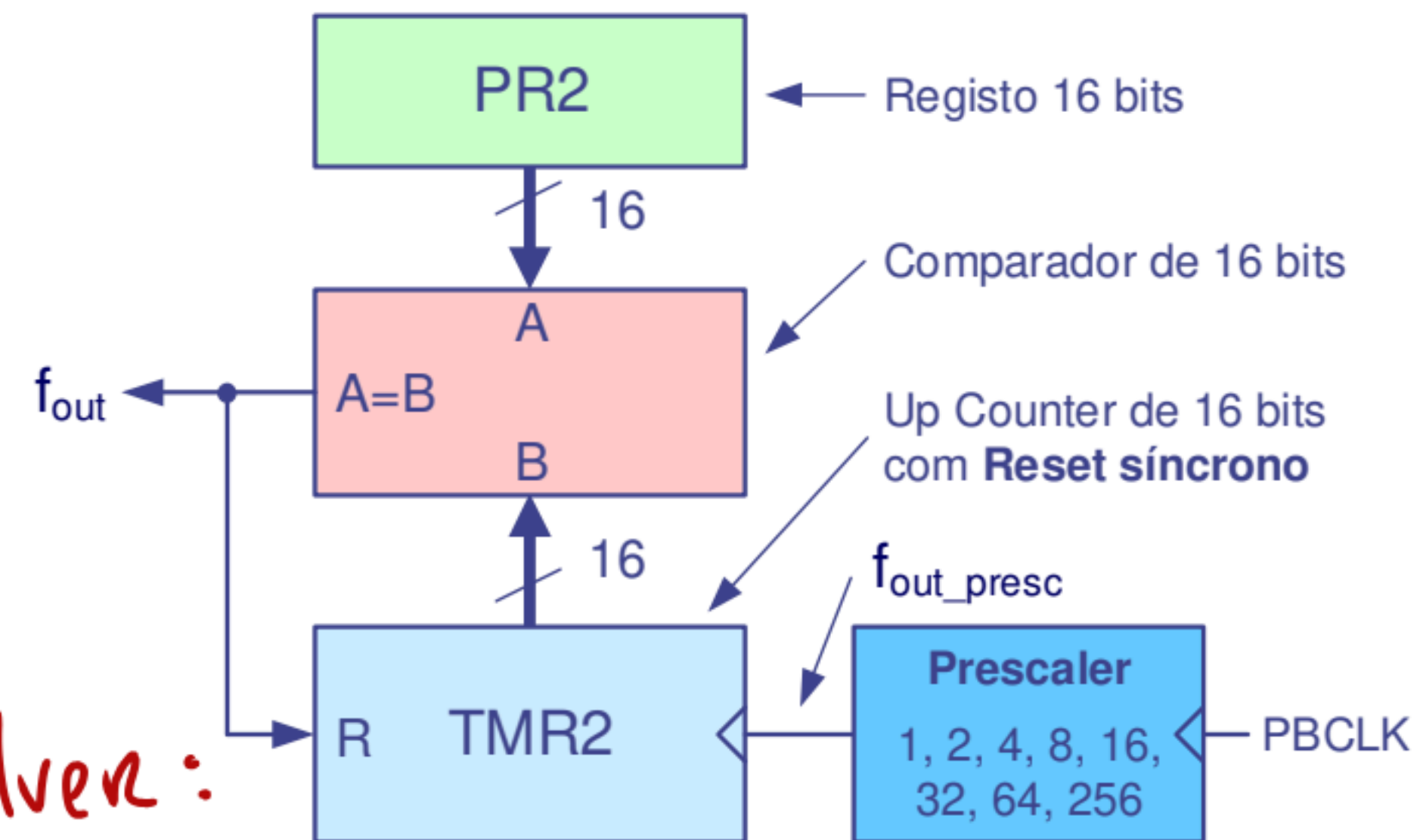
Exercício

Calcular as constantes para gerar um sinal PWM com uma frequência de 8 Hz e um *duty-cycle* de 20%, usando T2 como referência e OC1 como saída (PBCLK = 20 MHz)

$$f_{out} = \frac{f_{out_presc}}{PR2 + 1} = \frac{K_{prescaler}}{PR2 + 1}$$

$$K_{prescaler} = \frac{PBCLK}{(PR2 + 1) * f_{out}}$$

2 incógnitas, para resolver:



1. Cálculo da constante de divisão do *prescaler*

$$K_{prescaler} \geq \left\lceil \frac{2^{16} - 1}{(65535 + 1) * 8} \right\rceil = 39$$

← n existe vamos escolher o maior valor mais próxima

$$K_{prescaler} = 64$$

Valor máximo da constante PR2

Exercício (continuação)

2. Cálculo da constante de divisão do *timer* (PR2), com Kprescaler=64

$$f_{out_presc} = \frac{PBCLK}{K_{prescaler}} = \frac{20 * 10^6}{64} = 312500 \text{ Hz}$$

$$PR2 = \left(\frac{f_{out_presc}}{f_{out}} \right) - 1 = \frac{312500}{8} - 1 = 39062$$

8 freq que queremos na saída

3. Cálculo de OC1RS (*duty-cycle* de 20%):

$$OC1RS = \frac{(PR2 + 1) * dutyCycle}{100} = \frac{(39062 + 1) * 20}{100} = 7813$$

reset sincrono

3.1 Alternativa ao cálculo de OC1RS:

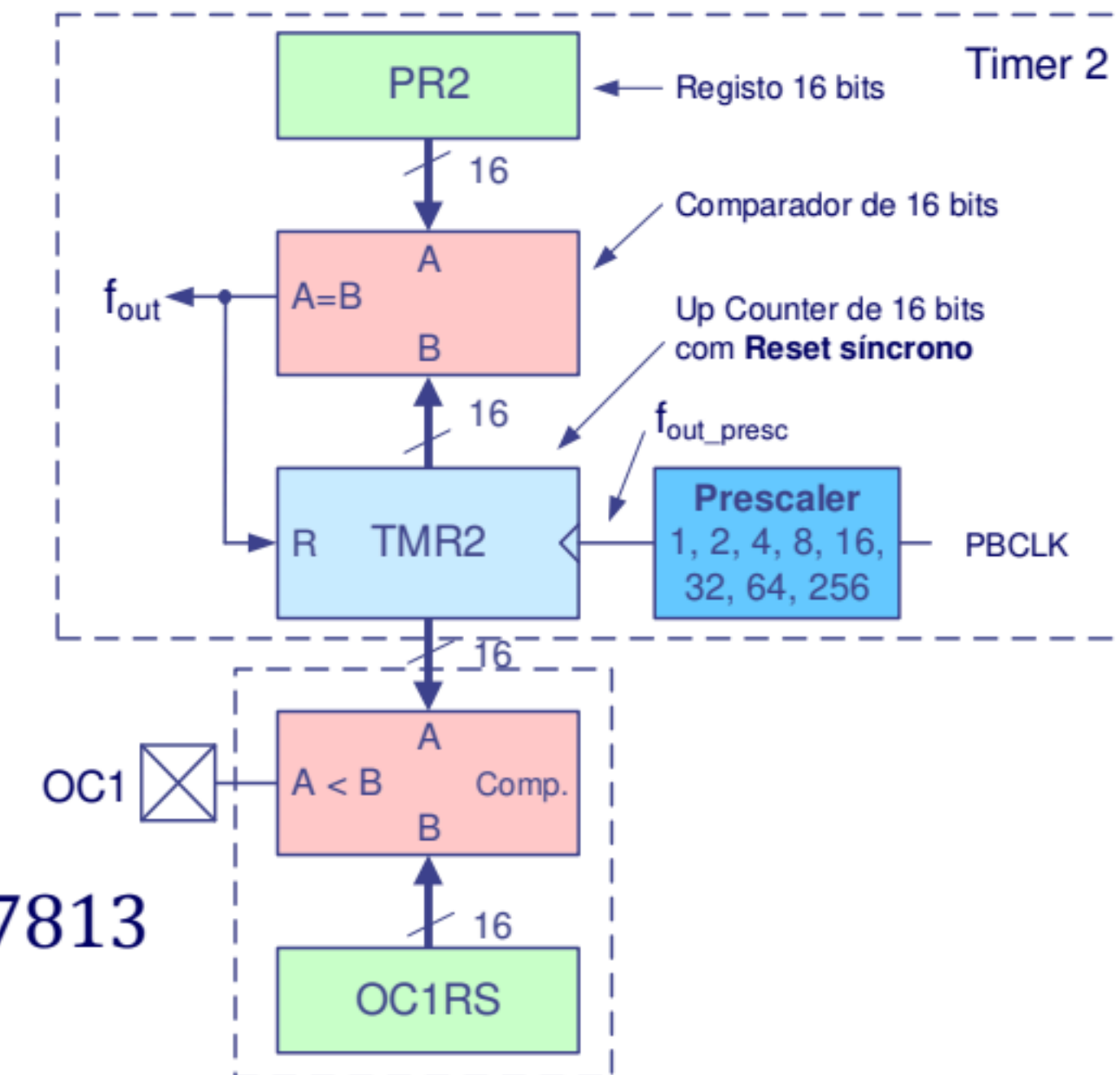
$$t_{ON} = 0.2 * T_{out} = \frac{0.2}{8} = 25 \text{ ms}$$

tempo a 1 (t_{ON}) do sinal de saída

$$T_{IN} = \frac{1}{f_{out_presc}} = \frac{1}{312500} = 3.2 \text{ us}$$

período do sinal à entrada do *timer*

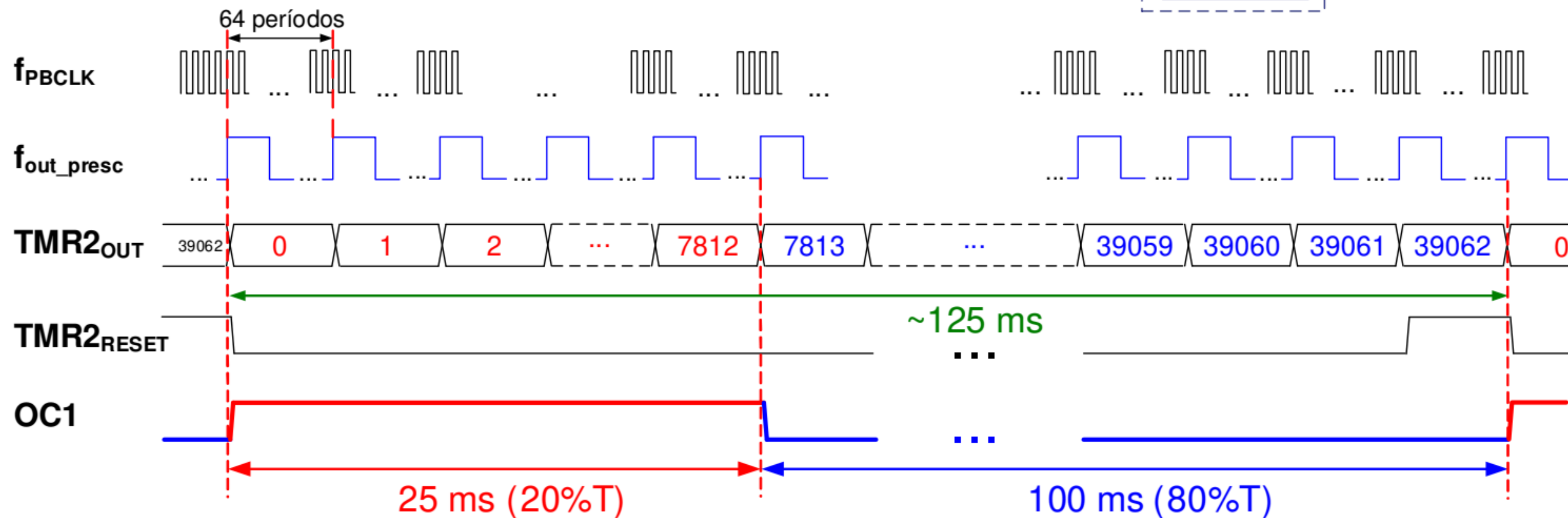
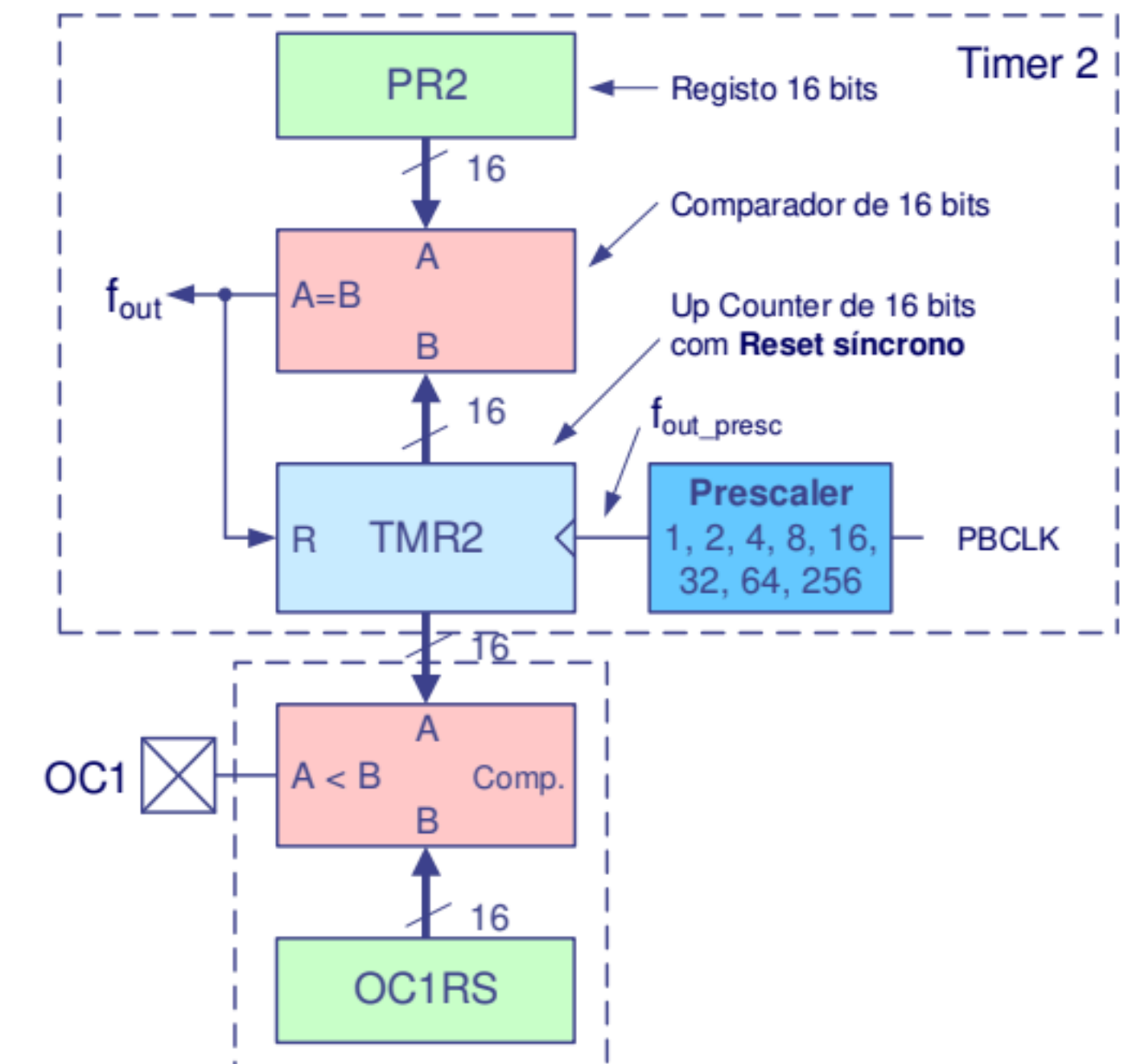
$$OC1RS = \frac{t_{ON}}{T_{IN}} = \frac{25 * 10^{-3}}{3.2 * 10^{-6}} = 7813$$



forma mais complexa

Exercício (comportamento temporal)

- Settings:
 - Pre-scale factor = 64
 - PR2 value = 39062
 - OC1RS = 7813
- $f_{PBCLK} = 20 \text{ MHz}$
- $f_{out_presc} = 312.5 \text{ KHz}$
- $TMR2(OC1)_{period} = 125 \text{ ms} (f=8\text{Hz})$
- Duty-cycle = 20%



PIC32 – Resolução do sinal PWM

- A resolução de um sinal PWM dá uma medida do número de níveis com que se pode variar o *duty-cycle* do sinal
- Pode ser definida como:
 - **Resolução = $\log_2 (T_{\text{PWM}} / T_{\text{IN}})$** PWM de 10 bits = 23 níveis
 - em que T_{PWM} é o período do sinal PWM gerado e T_{IN} é o período do sinal à entrada do gerador de PWM
- Para o caso do PIC32:
 - **Resolução = $\log_2 (T_{\text{PWM}} / (T_{\text{PBCLK}} * \text{Prescaler}))$** , ou, mais simplesmente:
 - **Resolução = $\log_2 (\text{PRx} + 1)$**
- Exercícios:
 - determine o valor das constantes PRx e OCnRS para a geração de um sinal com uma frequência de 100 Hz e 25% de *duty-cycle*, supondo PBCLK = 20 MHz, maximizando a resolução do sinal PWM; determine o valor das constantes para um *duty-cycle* de 80%
 - determine a resolução do sinal PWM que obteve; determine a resolução do sinal de PWM do exemplo do slide anterior

Exercícios

1. Pretende-se gerar um sinal com uma frequência de 85 Hz. Usando o Timer T2 e supondo PBCLK = 50 MHz:
 - calcule o valor mínimo da constante de divisão a aplicar ao *prescaler* e indique qual o valor efetivo dessa constante
 - calcule o valor da constante PR2
2. Repita o exercício anterior, supondo que se está a usar o Timer T1
3. Pretende-se gerar um sinal com uma frequência de 100 Hz e 25% de "duty-cycle". Usando o módulo "output compare" OC5 e como base de tempo o Timer T3 e supondo ainda PBCLK = 40 MHz:
 - determine o valor efetivo da constante de *prescaler* que maximiza a resolução do sinal PWM
 - determine o valor das constantes PR3 e OC5RS
 - determine a resolução do sinal de PWM obtido